



THE AI AGENT FACTORY
— By Panaversity —

Workflow Design & Diagnosis

Which steps AI should own, and what to do when it goes wrong.

A hands-on crash course. Bring one real workflow your team runs every week.

Two teams, one tool

Same product, same process, same model. Both adopted AI for contract review.

The first team mapped the work step by step

AI extracted the clauses, flagged departures from the playbook, drafted the redline. A lawyer approved every change.

Review time dropped by about half. Quality held.

The second team pointed AI at the whole process

It drafted well, so they let it approve low-risk clauses unsupervised.

Within a month an approved clause created an obligation nobody caught. The tool was pulled.

The difference was which steps each team chose to hand over.

The output is not the verdict

A good result does not prove a step was safe to hand over. A bad one does not prove the work cannot be done.

What tells you is the structure underneath. Four things decide:

Which step

A step is decided in advance.
A session is judged afterwards.

What it costs

In the bad case, not on average.

Who answers

Accountability never moves onto the machine.

When it started

Timing tells you where to look first.

Judge the step by what it costs and who answers for it. Judge a failure by when it started.

The whole course in one minute

Six questions, in the order you will need them.

Stage	The question that decides it
Design	What are the actual requirements, and which numbers must be computed?
Map	For this step: can it be undone, what does an error cost, who answers?
Build	Does this still fit in a prompt-and-iterate loop, or has it outgrown it?
Diagnose	When did the symptom first appear?
Persist	Is this fix a rule, a reference, or a procedure?
Communicate	What does it do, and where exactly is the human gate?

The rest of the session is how to answer each one well.

Try it now: redesign your own workflow

Five minutes. Pick a real workflow with more than three steps and at least one decision.

Paste this

Here is a workflow my team runs every week: [list the steps in order, roughly, in your own words].

Redesign it with AI inside. For each step, tell me whether AI should do it, and what the new workflow looks like.

Find the step that cannot be undone

The payment that goes out. The message that reaches the customer.
The document that gets signed.

Then ask it why

Why did you keep that step human? What specifically made you decide that, and how sure are you?

The information it was missing is not information about AI. It is information about your work, and only you have it.



THE AI AGENT FACTORY
By Panaversity

Part 1 of 6

Design the work before you place the tool

Requirements first. Then the numbers. Then the window test.

From a messy brief to requirements

Extraction is the first pass, not the trustworthy one. Ask for structure, then attack it.

Prompt 1 • Extract

From the attached RFP and the email thread, extract every distinct requirement we are being asked to address. For each one give me a short label, the exact section it comes from, whether our thread already answers it, and whether it is ambiguous enough to need a clarifying question. Return it as a table.

Prompt 2 • Attack

Review the requirements you extracted. Which are ambiguous as written? Which could two people on our team interpret differently? Which imply a requirement the document states only indirectly?

The tell: “delivered promptly after month end” is not a requirement. If you cannot write a check for it, it is not decidable yet.

An ambiguity caught here costs a clarifying question. Caught after the work is built, it costs the work.

Computed, never written

Models can add. The problem is that you cannot tell from the page whether they did.

“If a result matters, compute it. Never write it.”

Synthesis it does well: options, trade-offs, many factors at once.

Calculation goes to code execution, so each line traces to something you can re-run.

Paste this

Using code execution on the attached ticket data, calculate quarterly volume growth and average tickets resolved per analyst. Then, from those figures, recommend the headcount needed to hold our current resolution time next quarter, and show the assumptions.

Computation first, recommendation second, assumptions shown.
That is what lets somebody disagree without redoing the arithmetic.

When numbers come back subtly wrong, the cause is usually that a calculation was asked for in prose.

The window test

A capacity plan recommended hiring three analysts. The team had been under a hiring freeze for two weeks.

Synthesis steps: inside the window

The ticket data. The growth trend. The throughput per analyst. The documented options. Everything they need can be put in front of the model.

Judgement steps: outside it

The hiring freeze. Risk appetite. Which department the CFO is protecting. A model with unavailable inputs does not stop; it fills the gap at full confidence.

**“Could I put everything this step needs into the window?
If not, the step is mine.”**

Research is a synthesis step too. For anything current, priced, or dated, ask for a source you can open, never a confident answer.

Passing the window test makes a step eligible for AI. It never makes it owned by AI.



THE AI AGENT FACTORY
By Panaversity

Part 2 of 6

The delegation map

Every step takes three decisions. Mixing them builds a map that looks reasonable and is wrong.

Three decisions per step

In this order, and never backwards.

1 • Eligibility

May AI touch this at all?

Two gates. May this data enter this tool?
Can everything the step needs be supplied? A no at either ends the question.

2 • Ownership

Who owns it?

Reversibility, stakes, accountability.
AI-appropriate, human-retained, or collaborative.

3 • Implementation

What carries it?

A Skill, Project knowledge, code execution, a standing instruction, or a named review gate. Decided last.

The row teams skim: a human-retained step is not finished when you write “human” next to it. It needs a named person, a named moment, and a named thing they are checking.

“Must be computed” belongs under what carries it. It is never a reason to hand the step to AI.

Three criteria decide ownership

Only these three. How well AI did on your test run is deliberately not one of them.

Reversibility

Can it be undone? A draft can be rewritten. A sent email cannot be unsent. For a reversible step the cost of an error is a redo. For an irreversible one, the cost is the error.

Stakes

What does a mistake cost in the bad case, not on average? A misfiled label costs a minute. A miscalculated penalty exposure costs the penalty.

Accountability

Is this step the call somebody answers for, or an input they judge before it acts? Inputs can be delegated. The decision itself cannot, because nothing downstream is left to carry it.

Three questions, not a score. Usually one decides the step and the other two agree. Name that one; it is the column an auditor reads.

Where they disagree, the strictest wins. A reversible, low-stakes step that is still the decision somebody answers for stays human.

Accountability never moves onto the machine, because the machine cannot be asked.

Worked map: contract review

AI does real work here, including the redline. The human owns the decisions and the irreversible actions.

Workflow step	Owner	Criterion that decided it	Carried by
Extract clauses from the contract	AI	Reversible, low stakes	Skill
Flag departures from the company playbook	AI	Reversible, and errors surface at the redline	Skill
Draft the redline and the rationale	Collaborative	High stakes, so a human judges each edit	Skill + gate
Compute financial exposure of a penalty clause	AI	Reversible, and checked at the approval gate below	Code execution
Approve or reject each change	Human	This step is the decision somebody answers for	Named reviewer
Sign and send	Human	Irreversible and externally binding	Named signer

Take the approval gate away and the penalty-exposure step stops being AI-appropriate, even though the arithmetic has not changed.

Worked map: onboarding documents

The same three criteria on work that has nothing to do with law.

Workflow step	Owner	Criterion that decided it	Carried by
Pull new-hire details from the HRIS export	AI	Reversible, and checked at the confirmation step	Code execution
Draft the offer letter from the approved template	AI	Reversible draft, low stakes until it is sent	Skill
Personalise the welcome note	Collaborative	The manager's voice cannot be supplied	Named reviewer
Confirm compensation matches the approved req	Human	This step is the check, nothing follows it	Named reviewer
Send the signed offer	Human	Irreversible and externally binding	Named signer

Mechanical and draft steps delegate. The figure-confirmation step and the irreversible action stay human. Nobody decided that by taste. The criteria decided it, twice, in two domains.

A map produced by three questions is one your colleague produces the same way, and one you can defend to an auditor a year later.

Try it now: find the step you cannot undo

Five minutes. Same workflow as the opening, now that you have the criteria.

Paste this

Here is a workflow: [your steps].

For each step, answer three questions and nothing else. Can it be undone? What does an error cost in the bad case, not the average case? Is this step the decision somebody answers for, or an input that somebody else judges afterwards? Do not recommend anything yet.

Then classify each step yourself

AI-appropriate, human-retained, or collaborative. Write the criterion that carried each row, by name.

For every collaborative step

Who specifically reviews it, at what moment, checking what? If you cannot name a person and a moment, that step is automated.

Ask for the analysis and the recommendation together and you get a recommendation with analysis written to support it.

Three ways a map goes wrong

All three look reasonable at the moment they happen.

Halo delegation

A step gets handed to AI because the previous step went well. The competence was real. It just belonged to a different step.

The unstaffed gate

Review becomes a glance, then a click. Nobody changed the map, but the step is now automated. A collaborative step with no real reviewer is an automated step.

Mapping the tool, not the work

The team built a good Skill, so the workflow acquires a step shaped like it. Hardest to spot from inside, because the tool genuinely is used well.

The order to check them in: map the work with no tools in mind, classify each step independently, then ask of every collaborative step who specifically is reviewing and when.

Drafting quality is evidence about the draft. It is not evidence about the decision.

Spot the three problems

A customer-complaint workflow at a mid-size software company. Two minutes, then we compare.

#	Workflow step	Owner	Stated reason
1	Pull the complaint and the customer’s ticket history	AI	Mechanical retrieval
2	Summarise the complaint into our issue template	AI	Our summarising Skill is very good at this
3	Classify severity (low, medium, high)	AI	The summaries have been accurate for three months
4	Draft the customer response	Collaborative	AI drafts, the duty agent reviews before it goes out
5	Send the response	AI	Low severity only, and the drafts have been strong
6	Log the outcome in the CRM	AI	Mechanical write

Bad maps are not built from false statements. They are built from true statements about the wrong step.

Part 3 • The loop, and the cycle that fails quietly

Ideate, prototype, feedback, refine. Run it inside a Project. A worked example: three cycles, no code written.

Cycle 1

Build

“Build a simple dashboard artifact that shows these five metrics, with a chart for each.” Works. You have not yet asked for anything hard.

Cycle 2

Filter and totals

The filter works. The totals are wrong, and not obviously: written rather than computed. The fix is not a better prompt. The totals row belongs in code execution.

Cycle 3

Colour and print

Brand palette and a print-friendly layout. Works first time. A description problem, and describing it better solved it.

Escalation signal. Six months later three departments open the dashboard every Monday. Dependency means uptime, access, someone to call: developer or architect work. Escalating is a correct answer, not an admission.

Telling a description problem from a feature problem is the skill. Part 4 is the procedure for doing it.

Part 4 • Four causes, told apart by timing

When output disappoints, ask one question before changing anything: when did the symptom first appear?

When the symptom appeared	First hypothesis	Cheap confirmation	Fix if confirmed
Wrong from the first answer	Under-specification	Re-read the prompt. Is the missing thing actually in it?	Add what it left out
Started fine, then degraded	Context overload	Restate the instruction in a fresh session. Does it hold?	Restart or summarise
One repeatable error type	Wrong feature or model	Run the step once with the right feature or a stronger tier	Change the feature or the tier
Used to work, now does not	Stale configuration	Open the instruction or knowledge source and check its date	Maintenance on the configuration

Timing tells you where to look first. It does not tell you what the cause is.

The diagnostic sequence, cheapest first

Run these in order, before concluding the task is not a fit. Each rung is a hypothesis plus its confirmation.

1 Re-read the prompt against its five components: role, context, task, constraints, format.

2 Check the conversation length. Restart or summarise?

3 Check the feature and the model. A calculation in prose? A complex task on a speed tier?

4 Check the configuration. Are the instructions, knowledge, and Skills current?

5 Only then question whether the task fits. Reached first it is an excuse. Reached last, a conclusion.

Paste this next time an output disappoints

Before we fix this, help me diagnose it. I will describe what happened and when. Ask me one question at a time to work out whether this is under-specification, context overload, a wrong feature or model, stale configuration, or a task that does not fit. Do not suggest a fix until you have a diagnosis.

Reach for the sequence the moment output disappoints, before forming an opinion about whose fault it is.

Part 5 • Turn a reaction into an instruction

A reaction names how the output feels. An instruction names what to change so the next output is different.

Reaction	Instruction	Which lever
“Too generic”	Name the audience and the one action I want them to take	The prompt
“Wrong tone”	Add a tone constraint that applies to every draft	The instruction
“Missed the point”	State the single question the output must answer, up front	The prompt
“It’s using old data”	Replace the source document in the knowledge base	The knowledge

“What specifically would have to be present for this to be right, and which part of the setup controls that?”

If you cannot name the lever, the critique is still a reaction, and the next attempt will be a guess that looks like a revision.

Rule, reference, or procedure

If a correction will be needed again, by you or by someone else, it belongs in configuration.

A rule changes behaviour: standing instruction

“Always state the target segment in the first line.”

A reference changes what is known: knowledge base

The brand voice guide. The current product list.

A procedure changes how the work is done: Skill

“Generate the weekly report in this exact format, in this order.”

Paste this, for a correction you have made more than twice

I keep making this correction: [the correction].

Is this a rule, a reference, or a procedure? Then write it in the form its home needs: a standing instruction, a knowledge-base entry, or the steps of a Skill.

An analyst typed “exclude cancelled orders” into the chat every month. He took two weeks off, a colleague ran the report, and the cancelled orders went out.

The fix existed the whole time. The failure was leaving it somewhere only one person could find.

Measure the before

The step almost everyone skips. Without a baseline you have an improvement you believe in and cannot state.

One unchanged run, three numbers

Time

per run

Rounds

of revision

Steps

done by hand

Run old and new in parallel for two or three cycles. It buys evidence on your metric, a fallback if it is not better, and the agreement of the people whose work changed.

Be able to answer what it costs. Twenty minutes a week across three analysts is roughly fifty hours a year.

If the workflow is...	Optimise for	Because
An internal draft	Time	Speed is the point, and small errors are cheap
A customer-facing report	Consistency	Varying formats erode trust faster than delay
A compliance or finance output	Accuracy	One wrong figure costs more than an hour saved
Work done by several people	Variance	The cost lives between people, not within a person

You cannot reconstruct a baseline afterwards. Your estimate of the old way will be wrong in the direction that flatters you.

Part 6 • Say what it does, then name the gate

The same contract workflow, described three ways. The gate does not change. The detail each audience needs does.

To the legal lead

“It pulls the clauses out, marks departures from our playbook, and writes the first redline. Approving a change is still yours. The one to watch is an obligation implied rather than stated, which it can miss.”

To the practice executive

“Turnaround on a standard contract has gone from two days to about half a day, with the same approval standard. A lawyer still signs off every change before it leaves the building.”

To the client’s risk function

“Drafting is AI-assisted. A qualified lawyer reviews and approves every term, and nothing is sent without that sign-off.”

Paste this, with your own workflow

Here is an AI-assisted workflow my team runs: [your workflow, including the review step]. Write three descriptions: one for a technical colleague, one for an executive, one for a risk function. Each must name the human checkpoint. Do not use “fully automated”, “handles”, or “as good as a person”.

Stakeholders trust an AI workflow more, not less, when the human checkpoints are explicit.

The one-page map: copy this

The artifact you hand to a colleague, an auditor, or whoever takes the workflow over.

Header

Delegation map: [workflow name]

Owner: one named person

Last reviewed • **Next review:** dates

Steps

One row per step: **step** • **owner** • **criterion** • **carried by** . The criterion is reversibility, stakes, or accountability only.

Baseline

Measured before any change: time per run, revision rounds, manual steps. The metric this workflow optimises for.

Eligibility

Data gate: may this data enter this tool? Who confirmed, and when?

Input gate: can everything each step needs actually be supplied?

Gates

One line per collaborative or human step: reviewed by [name], at [when], checking [what specifically].

If it goes wrong

Stopped by whom, without permission? **Scope check:** how many outputs were affected? **Told:** who, within what time. **Map change** recorded, with a date.

A map without the owner, the next review date, and what each gate checks describes a decision. With them, it describes an operation.



THE AI AGENT FACTORY
By Panaversity

The sentence to carry

The output is not the verdict.

**Judge the step by what it costs and who answers for it.
Judge a failure by when it started.**

Homework: one workflow, all six parts, about an hour. Map it, break it, fix it, and write the three descriptions. Next in Foundations: Governance, Risk & Responsible Use.